

Module: Technologie Web

Licence MIP Semestre 3

PR SARA OUAHABI

MAIL: S3.OUAHABI@GMAIL.COM

ANNÉE UNIVERSITAIRE: 2024-2025

Partie 2:

Développement Web avec CSS

Chapitre 5:

Création d'un contenu flexible, adaptatif et animé

Objectifs

- FlexBox
- Cadres avec <iframe>
- Responsive design avec Media queries
- Animation

Utilisation des conteneurs Flex (FlexBox)

Pourquoi utiliser Flexbox ?

Avant l'insertion de la méthode Flexbox, il existait quatre modes de mise en page :

- **Block**, pour les sections d'une page Web
- **Inline**, pour le texte
- **Tableau**, pour les données de tableau à deux dimensions
- **Position**, pour la position explicite d'un élément

La méthode **Flexbox** facilite la structuration et la mise en page souple et réactive, sans utiliser les marges et le positionnement.

La page est décomposée en un agencement de boîtes qui peuvent se suivre (boîtes voisines) et s'imbriquer (boîtes ancêtres).

En définissant les tailles et les interactions entre les différentes boîtes, on obtient une adaptation automatique des éléments de la page lorsqu'elle est redimensionnée.

Comment utiliser Flexbox ?

Le conteneur **flex** (**flex container**) ou bien la boîte dite « parent », contient les éléments **flex** (**flex items**) ou bien les boîtes « enfants ».

L'idée est définir, en utilisant les différentes propriétés CSS, l'ordre des "boîtes enfants" dans la "boîte parent".

Définir le conteneur flex (flex container)

Le conteneur est une balise html qui contient d'autres balises html.

Dans l'exemple ci-dessous, La conteneur correspond à la balise <div> pour laquelle on a défini la classe « **.conteneur** ».

Cette balise <div> contient deux balises <div> et une balise <figure>. Ces dernières deviennent alors des "éléments flex" (flex items) mis en évidence ici par la classe "**.element**".

```
<div class="conteneur">  
  <div class="element">Élément 1</div>  
  <figure class="element">Élément 2</figure>  
  <div class="element">Élément 3</div>  
</div>
```

La boîte se transforme en "conteneur flex" dès lors qu'on lui attribue la propriété CSS "flex" comme montré dans le code CSS suivant

```
.conteneur {  
  display : flex;  
}
```

Définir les axes du conteneur flex avec la propriété flex-direction

Axe principal : main axis

Le conteneur flex possède un axe principal (main axis) sur lequel les éléments flex peuvent se suivre.

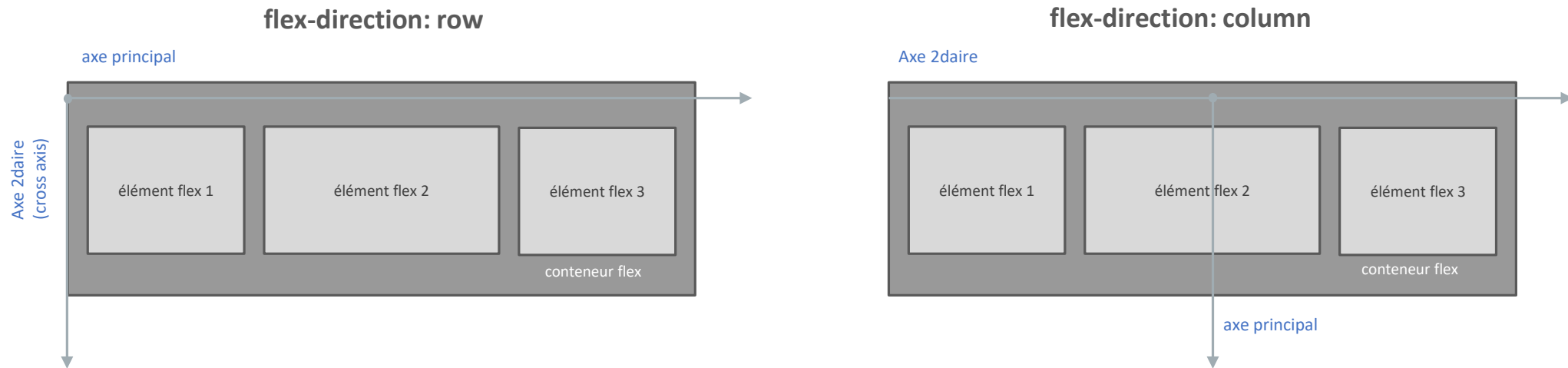
L'axe principal est défini par la propriété **flex-direction** qui peut prendre les valeurs suivantes:

- **row** : les éléments flex se suivent sur une ligne, ce qui correspond à la direction normale du sens d'écriture du document
- **row-reverse** : les éléments flex se suivent sur une ligne en ordre inversé. ce qui correspond à la direction inverse du sens d'écriture du document
- **column** : les éléments flex se suivent sur une colonne.
- **column-reverse** : les éléments flex se suivent sur une colonne en ordre inversé

Définir les axes du conteneur flex avec la propriété flex-direction

Axe secondaire : cross axis

Le choix de l'axe principal impose la direction de l'axe secondaire (**cross axis**) qui lui est toujours perpendiculaire.



Le choix des axes est essentiel car leur orientation va déterminer le résultat de l'application des propriétés suivantes :

justify-content : définit comment les éléments flex sont positionnés le long de l'axe principal ;

align-items : définit comment les éléments flex sont positionnés le long de l'axe secondaire ;

align-self : définit comment un seul élément flex est positionné le long de l'axe secondaire.

Positionner les éléments à l'intérieur d'un conteneur flex

Les éléments flexibles peuvent être alignés, justifiés et distribués le long des axes de leur conteneur grâce à différentes propriétés.

Dans l'exemple suivant, on a choisi une direction "**row**", ce qui signifie que l'axe principal est l'axe horizontal et que l'axe secondaire est l'axe vertical.

```
<div class="conteneur">
  <div class="element1">élément 1</div>
  <div class="element2">élément 2</div>
  <div class="element3">élément 3</div>
  <div class="element4">élément 4</div>
</div>
```

```
.conteneur {
  background-color : blue
  height : 230px;
  width : 500px;
  display : flex;
  flex-direction : row;
  align-items : stretch | flex-start | flex-end | center | baseline;
  justify-content : flex-start | flex-end | center | space-around | space-between |
space-evenly;
}
```

Aligner le long de l'axe principal : la propriété "justify-content"

La propriété "**justify-content**" permet d'aligner les éléments le long de l'axe principal dans la direction définie par "flex-direction".

Les valeurs de la propriété "**justify-content**" : **flex-start**, **flex-end**, **center**, **space-around**, **space-between** et **space-evenly**.

Exemple 1 : justify-content : flex-start => les éléments flexibles sont placés à partir de la ligne de début du conteneur sur l'axe principal.

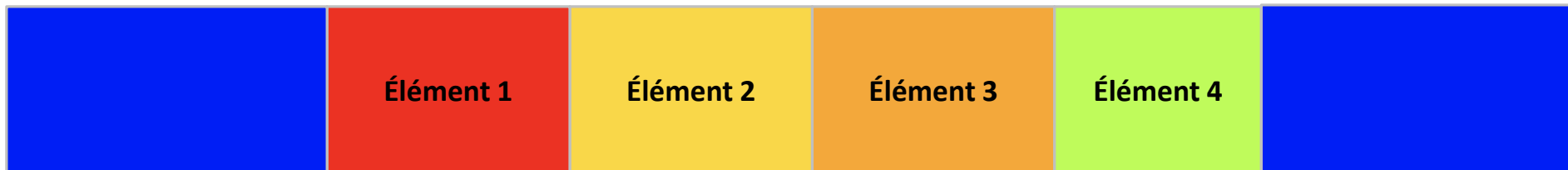


Aligner le long de l'axe principal : la propriété "justify-content"

Exemple 2 : justify-content : flex-end => les éléments flexibles sont placés à partir de la ligne de fin du conteneur sur l'axe principal.



Exemple 3 : justify-content : center => les éléments flexibles sont centrés le long de l'axe principal.



Aligner le long de l'axe principal : la propriété "justify-content"

Exemple 4 : justify-content : space-between => l'espace disponible est réparti de façon égale entre chaque élément.

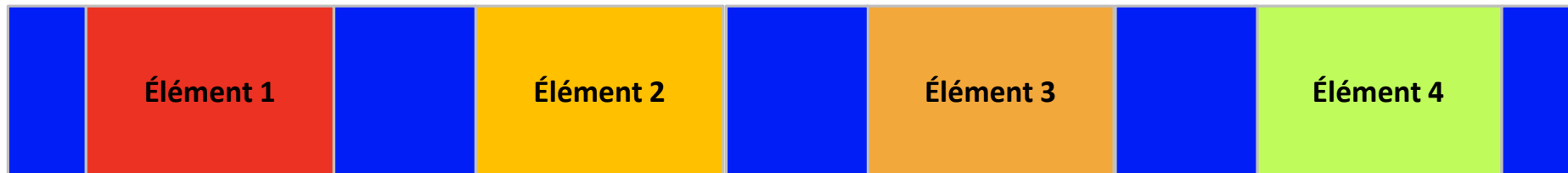


Exemple 5 : justify-content : space-around => l'espace disponible est réparti de façon égale entre chaque élément, y compris au début et à la fin.



Aligner le long de l'axe principal : la propriété "justify-content"

Exemple 6 : justify-content : space-evenly => l'espace disponible est réparti de façon égale entre chaque élément avec un espace entier au début et à la fin.



Aligner le long de l'axe secondaire : la propriété "align-items"

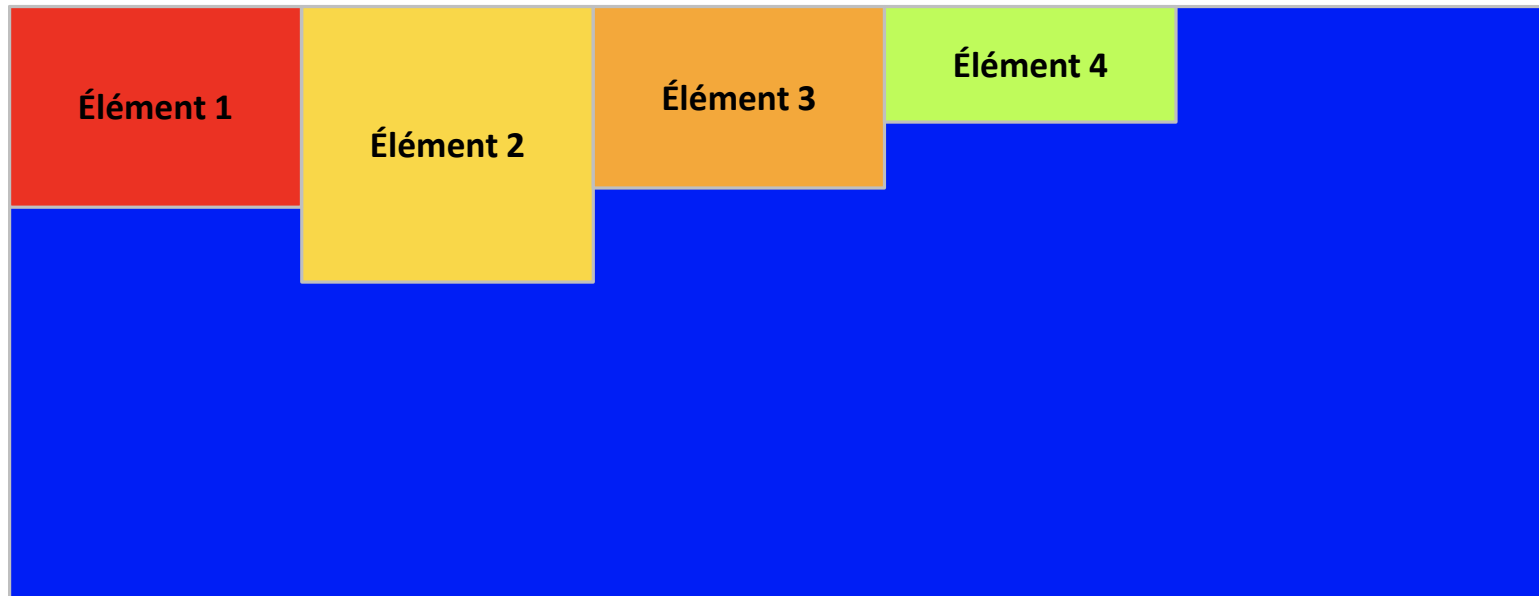
La propriété "align-items" peut prendre 5 valeurs : **stretch**, **flex-start**, **flex-end**, **center** et **baseline**.

Exemple 1 : align-items : stretch => les éléments flexibles sont étirés le long de l'axe secondaire.



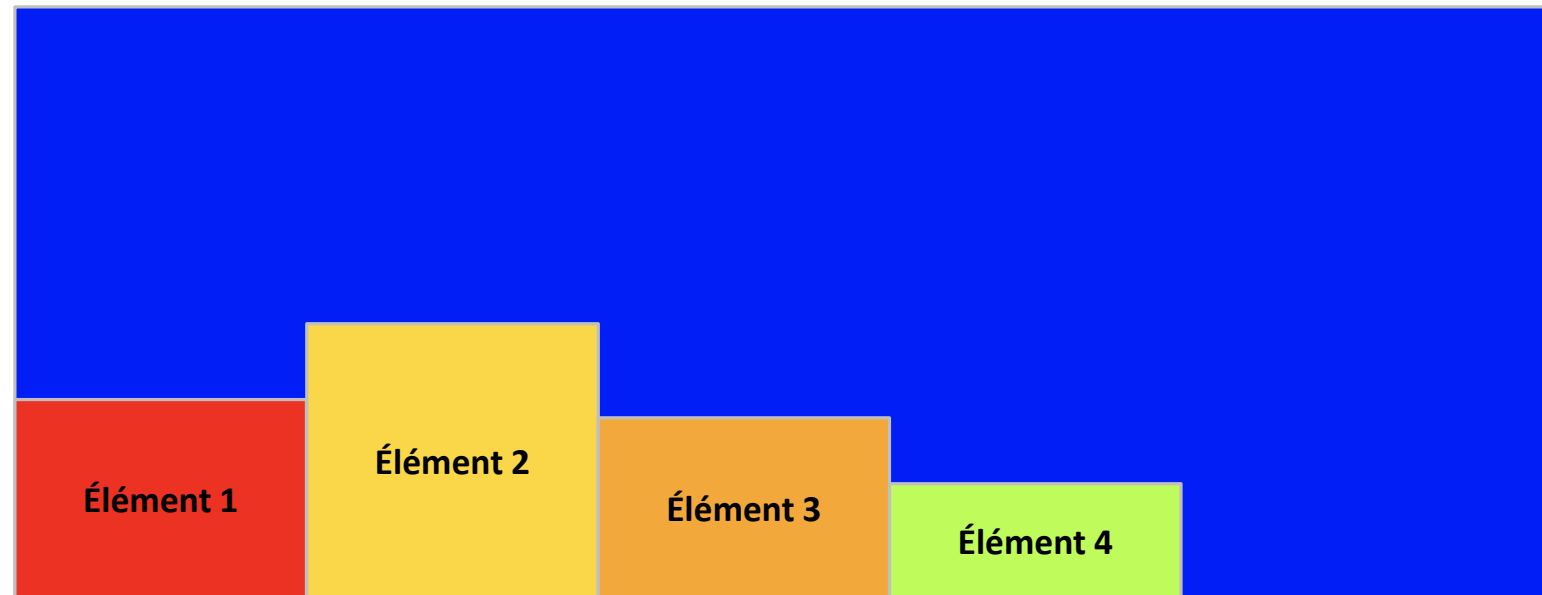
Aligner le long de l'axe secondaire : la propriété "align-items"

Exemple 2 : align-items : flex-start => les éléments flexibles sont alignés sur la ligne de début de l'axe secondaire.



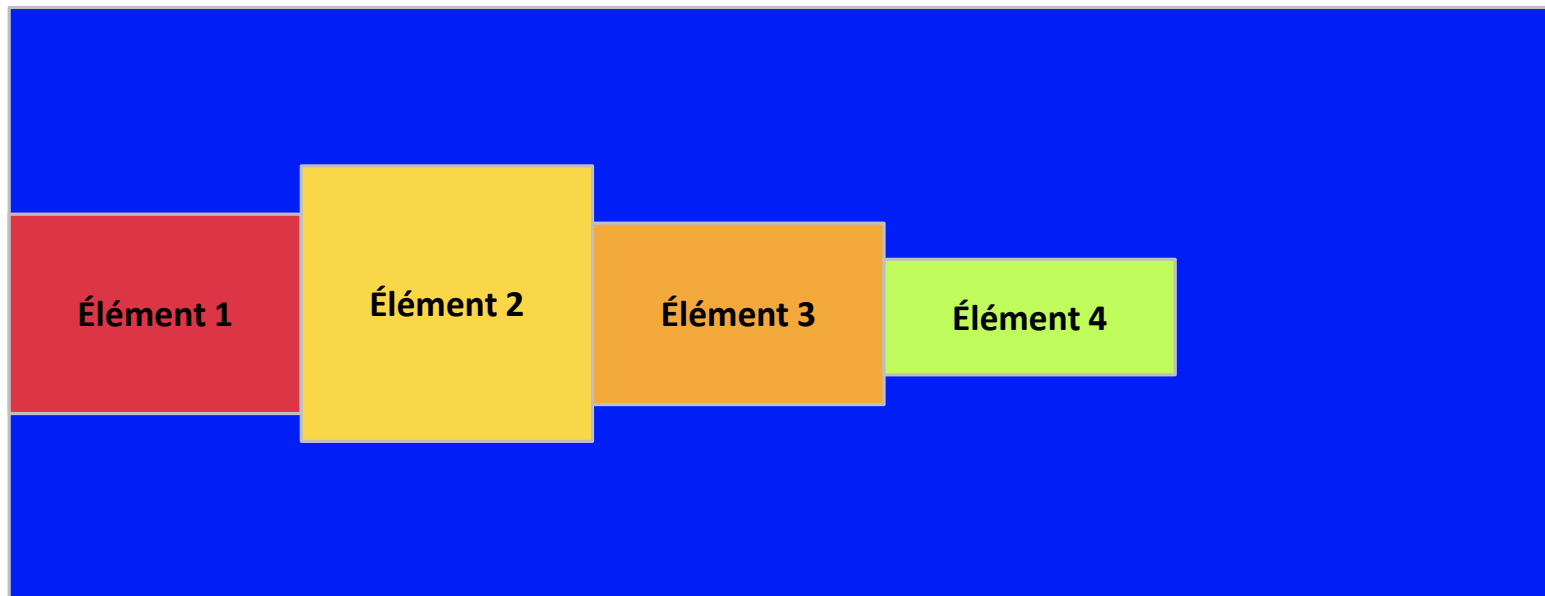
Aligner le long de l'axe secondaire : la propriété "align-items"

Exemple 3 : align-items : flex-end => les éléments flexibles sont alignés sur la ligne de fin de l'axe secondaire.



Aligner le long de l'axe secondaire : la propriété "align-items"

Exemple 4 : align-items : center => les éléments flexibles sont centrés sur la ligne d'axe secondaire.



Maitriser les passages à la ligne : la propriété "flex-wrap"

La propriété "**flex-wrap**" peut prendre les valeurs suivantes :

- **Flex-wrap : nowrap** - c'est la valeur initiale. Si les éléments flexibles sont trop larges pour tenir dans leur conteneur, ils débordent du conteneur
- **flex-wrap : wrap** - si les éléments flexibles sont trop larges pour tenir dans le conteneur, ils passent automatiquement sur la ligne suivante (du haut vers le bas)
- **flex-wrap : wrap-reverse** - si les éléments flexibles sont trop larges pour le conteneur, ils passent automatiquement sur la ligne suivante (du bas vers le haut)

La propriété raccourcie flex-flow

La propriété raccourcie **flex-flow** combine **flex-direction** et **flex-wrap**. Elle peut, par exemple, prendre les valeurs "row wrap" ou "column wrap".

Exemple

```
.container {  
  display: flex;  
  flex-flow: row wrap;  
}
```

Utilité des cadres

- Nous avons déjà vu comment on peut agencer les éléments HTML dans une page à l'aide des balises `<div>` et `<span>`.
- Nous avons pu aussi réaliser des menus intéressants.
- Supposons qu'on désire que ce menu soit repris sur toutes les pages du site:
 - si on réécrit le code nous allons le dupliquer et ce n'est pas intéressant.
- Supposons aussi qu'on clique sur un lien du menu et qu'on veuille que la page web correspondante s'affiche au centre.
- Le seul moyen de le faire en HTML est d'utiliser la balise **`<iframe>`**.

Utilité des cadres

- L'utilité de cette balise est de se comporter comme un conteneur pouvant afficher une page web locale ou distante.
- Pour les navigateurs ne supportant pas cette balise on peut ajouter du texte pour l'indiquer.
- On ne peut mettre que du texte entre les deux balises `<iframe>` et `</iframe>`.

Syntaxe de la balise <iframe>

- **Code HTML :**

<iframe

src="demo.html"

name="centre"

class= " nom du cadre">

Votre navigateur ne supporte pas les iframes

</iframe>

- Pour afficher une page locale dans ce cadre il suffit de mettre dans la propriété **target** le **nom** du cadre.

Exemple

Accueil

Formations

Recherche

Actualité

Contact

Bienvenue sur mon site

Exemple: Page index

- Première étape c'est de créer un fichier index.html où nous allons définir les deux cadres :

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="utf-8">
5     <link rel="stylesheet" href="cadre.css"/>
6     <title>exemple 61</title>
7   </head>
8   <body>
9     <iframe src="entete.html" scrolling="no" class="menu" name="menu"></iframe>
10    <iframe src="corps.html" scrolling="yes" class="central" name="centre"></iframe>
11  </body>
12 </html>
```

Exemple: Page entete

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="utf-8">
5      <title>page entete</title>
6      <link rel="stylesheet" href="cadre.css"/>
7  </head>
8  <body>
9  <table>
10 <tr>
11 <td><a href="accueil.html" target="centre">Accueil</a></td>
12 <td><a href="infos.html" target="centre">Formations</a></td>
13 <td><a href="recherche.html" target="centre">Recherche</a></td>
14 <td><a href="contact.html" target="centre">Actualité</a></td>
15 <td><a href="login.html" target="centre">Contact</a></td><td ></td>
16 </tr>
17 </table>
18 </body>
19 </html>
```

Exemple: Page corps

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="utf-8"/>
5      <title>page corps</title>
6  </head>
7  <body>
8      <h1> Bienvenue sur mon site</h1>
9  </body>
10 </html>
11
```

Exemple: Page corps

```
1 <!DOCTYPE HTML>
2 <html>
3 <head>
4   <link rel="stylesheet" href="cadre.css" />
5   <!--<link rel="stylesheet" media="screen and (max-width: 1280px)" href="mystyles.css" />-->
6   <meta charset="utf-8"/>
7 </head>
8 <body>
9   <h1>Page d'accueil</h1>
10  <p>
11    c'est le contenu de la page d'accueil qui s'affiche dans
12    le cadre nommé "centre"
13    quand je clique sur accueil
14  </p>
15 </body>
16 </html>
```

```
1  .menu{
2      position:absolute;
3      text-align:center;
4      background-color:rgb(210, 180, 222);
5      border-radius:10px 10px 10px 10px;
6      padding-bottom:6px;
7      left:10%;
8      width:80%;
9      height:6%;
10     border:none;
11 }
12 .central{
13     position:absolute;
14     left:10%;
15     top:10%;
16     width:80%;
17     height:100%;
18     background-color:rgb(215, 189, 226 );
19     border-radius:10px;
20     padding:2px;
21     border:none;
22 }
23 td{ width:20%;padding:5px; }
24 a{text-decoration:none; color : white; font-weight:bold; }
25 p{text-align:justify;}
```

Autres propriétés de la balise iframe

loading	eager lazy	Spécifie si un navigateur doit charger un iframe immédiatement ou différer le chargement des iframes jusqu'à ce que certaines conditions soient remplies
referrerpolicy	no-referrer no-referrer-when-downgrade origin origin-when-cross-origin same-origin strict-origin-when-cross-origin unsafe-url	Cet attribut permet de définir une politique de fonctionnalité pour notre iframe. choisir quelles fonctionnalités de l'iframe activer ou désactiver.
sandbox	allow-forms allow-pointer-lock allow-popups allow-same-origin allow-scripts allow-top-navigation	Il nous permet de limiter de ce qu'on peut faire à l'intérieur de e notre iframe Question de sécurité

Les media queries

Différents médias



Modèle	Définition (pixels)	Taille de l'écran (pouce)	Résolution (ppp)
iPhone 3GS	320×480	3,5	164
HTC Hero	320×480	3,2	180
Nokia N900	800×480	3,5	266
iPhone 4s	960×640	3,5	326
Galaxy Notes	1280×800	5,3	285
iPad 2	1024×768	9,7	132
Galaxy Tab 10,1	1280×800	10,1	150
Playbook	1024×600	7	170
Galaxy Tab 8,9	1280×800	8,9	170
iPad 3	2048×1536	9,7	264
Macbook Air	1440×900	13,3	128
iMac	2560×1440	27	109

Adapter une page web au dispositif d'affichage: Introduction au Responsive Design

- Les pages Web peuvent être consultées à l'aide de nombreux types d'appareils : ordinateurs de bureau, tablettes et téléphones. Ainsi, les pages Web ne doivent pas omettre d'informations pour s'adapter aux appareils plus petits, mais plutôt adapter leur contenu pour s'afficher correctement sur n'importe quel appareil utilisé
- La conception Web réactive (**Responsive Design**) rend la page Web compatible et adaptable à tous les appareils. Elle est basée uniquement sur HTML et CSS (elle n'est pas un programme ou un code JavaScript)



- Le Responsive web design consiste à utiliser les langages CSS et HTML pour redimensionner, masquer, réduire, agrandir ou déplacer le contenu d'une page pour s'adapter à l'écran d'affichage

Les media queries

La règle **@media**, introduite dans CSS2, permet de définir différentes règles de style pour différents types de médias.

Les **médias queries** dans CSS3 ont étendu l'idée des types de médias CSS2 : examiner la capacité du périphérique au lieu de chercher son type.

Les requêtes multimédias peuvent être utilisées pour vérifier de nombreuses propriétés, telles que :

- Largeur et hauteur de la fenêtre
- Largeur et hauteur de l'appareil
- Orientation (mode paysage ou portrait)
- Résolution

Les media queries

- L'attribut media permet de définir des règles qui doivent s'appliquer lors du chargement du fichier.
- On peut proposer plusieurs fichiers CSS : un par défaut et d'autres pour différentes résolutions
- Deux techniques pour le définition

Avec l'attribut media
Dans le fichier css

L'attribut media

```
<!DOCTYPE html>
<html>
<head>
<metacharset="utf-8" />
<!--cas normal-->
<linkrel="stylesheet" href="style.css" />
<!--Pour les tablettes qui ont une résolution inférieure à 1280px -->
<linkrel="stylesheet" media="screen and (max-width: 1280px) and (max-
height:800px)" href="tablette.css" />
<title>Media queries</title>
</head>
....
</html>
```

Chargement des règles directement dans la feuille de style

L'exemple suivant change la couleur d'arrière-plan selon la règle suivante :

- Si la largeur de la fenêtre est 480 px au minimum, la couleur est le bleu clair
- Si la largeur de la fenêtre est 200 px au minimum, la couleur est le lightgreen
- Si la largeur de la fenêtre est inférieure à 200 px, la couleur reste blanche

```
@media screen and (min-width : 200px) {  
  body {  
    background-color : rgb(144, 174, 238);  
  }  
}  
@media screen and (min-width : 480px) {  
  body {  
    background-color : lightgreen;  
  }  
}
```

Les règles

- Il existe de nombreuses règles permettant de construire des media queries.
- color, height, width: couleur, hauteur, largeur de la zone d'affichage (fenêtre)
- device-height: hauteur du périphérique
- device-width: largeur du périphérique
- orientation : orientation du périphérique (portrait ou paysage)
- On peut rajouter min-width signifie « largeur minimale », max-height « hauteur maximale »
- La différence entre width et device-width se perçoit surtout sur les navigateurs mobiles des smartphones.
 - `<metaname="viewport"content="width=device-width, initial-scale=1.0">`
 - Viewport depend du navigateur
 - zoom initial correspond à une simple division mathématique de device-width/ viewport.

Les règles

media : type d'écran de sortie

- screen : écran « classique »,
- handheld: périphérique mobile,
- print: impression,
- tv : télévision,
- projection : projecteur,
- all : tous les types d'écrans.

Combinaison des règles

Les règles peuvent être combinées à l'aide des mots suivants :

- only: « uniquement » ;
- and : « et » ;
- not : « non ».

Exemple

- /* Sur les écrans, quand la largeur de la fenêtre fait au maximum 1280px */
@media screen and (max-width: 1280px)
- /* Sur tous types d'écran, quand la largeur de la fenêtre est comprise entre 1024px et 1280px */

@media all and (min-width: 1024px) and (max-width: 1280px)

- /* Sur les téléviseurs */

@media tv

- /* Sur tous types d'écrans orientés verticalement */

@media all and (orientation: portrait)

Créer des animations: Manipulation des propriétés d'animation

La propriété **animation** (ou bien les propriétés détaillées correspondantes) est utilisée pour créer une animation CSS. Elle permet de configurer la durée, le minutage et d'autres détails de l'animation.

L'apparence visuelle de l'animation est définie en utilisant des règles CSS de mise en forme au sein de la règle **@keyframes**.

Les propriétés détaillées rattachées à la propriété raccourcie animation sont :

animation-delay : Cette propriété définit le délai entre le chargement de l'élément et démarrage de l'animation.

animation-direction : Cette propriété précise si l'animation doit alterner entre deux directions de progression (faire des allers-retours) ou recommencer au début à chaque cycle de répétition.

animation-duration : Cette propriété définit la durée d'un cycle de l'animation.

animation-fill-mode : Cette propriété indique les valeurs à appliquer aux propriétés avant et après l'exécution de l'animation.

Propriétés d'animation

animation-iteration-count : Cette propriété détermine le nombre de répétition de l'animation. Pour répéter une animation infiniment, On utilise la valeur **infinite**.

animation-name : Cette propriété déclare un nom à l'animation (utilisé comme référence à l'animation pour la règle **@keyframes**).

animation-play-state : Cette propriété permet d'interrompre (« pause ») ou de reprendre l'exécution d'une animation.

animation-timing-function : Cette propriété configure la fonction de minutage d'une animation.

Créer des animations: Types d'animation

CSS permet l'animation d'éléments HTML sans utiliser JavaScript

Une animation permet à un élément de passer progressivement d'un style à un autre en modifiant les propriétés CSS

Les types d'animations en CSS :

- Les transformations
- Les transitions

Types d'animation: Les transformations

Les transformations 2D

Les transformations CSS 2D permettent de déplacer, faire pivoter, mettre à l'échelle et incliner des éléments.

La méthode **translate()**

Exemple : déplacer l'élément <div> de 50 pixels vers la droite et de 100 pixels vers le bas par rapport à sa position actuelle :

```
div {  
  width : 300px;  
  height : 100px;  
  background-color : cyan;  
  border : 1px solid black;  
}  
div :hover{  
  transform : translate(50px,100px);  
}
```

La méthode **rotate()**

Exemple : faire pivoter l'élément <div> dans le sens des aiguilles d'une montre de 20 degrés :

```
div {  
  width : 300px;  
  height : 100px;  
  background-color : cyan;  
  border : 1px solid black;  
}  
div :hover{  
  transform : rotate(20deg);  
}
```

Types d'animation: Les transformations

La méthode `scale()`

Exemple : augmenter la taille de l'élément `<div>` de deux fois sur sa largeur et trois fois sa hauteur :

```
div {  
  width : 300px;  
  height : 100px;  
  background-color : cyan;  
  border : 1px solid black;  
}  
div :hover{  
  transform : scale(2,3);  
}
```

La méthode `skew()`

Exemple : incliner l'élément `<div>` de 20 degrés le long de l'axe X et de 10 degrés le long de l'axe Y :

```
div {  
  width : 300px;  
  height : 100px;  
  background-color : cyan;  
  border : 1px solid black;  
}  
div :hover{  
  transform : skew(20deg,10deg);  
}
```

Types d'animation: Les transformations

La méthode **rotateX()**

Permet de faire une rotation par rapport à l'axe X (on peut utiliser aussi **rotateY** et **rotateZ**).

```
div {  
  width : 300px;  
  height : 100px;  
  background-color : cyan;  
  border : 1px solid black;  
}  
div :hover{  
  transform : rotateX(60deg);  
}
```

Types d'animation: Les Transitions

Transitions CSS

Les transitions CSS permettent de modifier les valeurs des propriétés sur une durée donnée.

Exemple : un élément <div> rouge de 100 px * 100 px subit un effet de transition pour la propriété width, d'une durée de 2 secondes :

```
div {  
  width : 100px;  
  height : 100px;  
  background : red;  
  transition : width 2s;  
}
```

Lorsqu'un utilisateur passe la souris sur l'élément <div>, une nouvelle valeur pour la propriété **width** s'applique. L'effet de transition démarre lorsque la propriété CSS spécifiée (largeur) change de valeur.

```
div :hover {  
  width : 300px;  
}
```

```
div {  
  transition :width 2s,  
  height 4s;  
}
```

Créer des animations: Utilisation des keyFrames

La règle @keyframes

La spécification d'un style CSS à l'intérieur de la règle @keyframes permet à une animation de passer progressivement du style actuel au nouveau style.

Exemple 1 :

```
@keyframes example {  
  from {background-color : red;}  
  to {background-color : yellow;}  
}  
  
/* The element to apply the animation to */  
div {  
  width : 100px;  
  height : 100px;  
  background-color : red;  
  animation-name : example;  
  animation-duration : 4s;  
}
```

définit le nom de l'animation

définit la durée de l'animation

Créer des animations: Utilisation des keyFrames

La règle @keyframes

Exemple 2 :

```
@keyframes example {  
  0%   {background-color : red;}  
  25%  {background-color : yellow;}  
  50%  {background-color : blue;}  
  100% {background-color : green;}  
}  
/* The element to apply the animation to */  
div {  
  width : 100px;  
  height : 100px;  
  background-color : red;  
  animation-name : example;  
  animation-duration : 4s;  
}
```

Créer des animations: Utilisation des keyFrames

La règle @keyframes

Exemple 3 :

```
@keyframes example {  
  0%   {background-color :red; left :0px; top :0px;}  
  25%  {background-color :yellow; left :200px; top :0px;}  
  50%  {background-color :blue; left :200px; top :200px;}  
  75%  {background-color :green; left :0px; top :200px;}  
  100% {background-color :red; left :0px; top :0px;}  
}
```

```
div {  
  width : 100px;  
  height : 100px;  
  position : relative;  
  background-color : red;  
  animation-name : example;  
  animation-duration : 4s;  
}
```

Moduleprojetweb@gmail.com ====30/11/2024